7

RUNTIME ENVIRONMENT AND VIRTUAL PROCESSOR ANOMALIES

In the previous three chapters, you've seen how malware can query and enumerate OS artifacts and configurations to understand its environment and detect that it's being analyzed. This chapter will focus on how malware can actively identify analysis sandboxes and VM environments by inspecting the anomalies that malware analysis tools introduce, monitoring virtual processor performance and timing, and abusing virtual processor instructions.

Detecting Analysis and Runtime Anomalies

When malware is executed in a sandbox or malware analysis environment, the sandbox or analysis tools can give away their presence in several ways. Sandboxes sometimes rename the malware file to a generic filename, for example. Sandboxes and analysis tools might also inject code into or modify code in the malware sample to better intercept and analyze the malware's actions. Some malware variants can detect these anomalies in their runtime environment. Let's look into these techniques in more detail.

Run Paths, Filenames, and Arguments

When malware is sent to an automated sandbox to be detonated and analyzed, it's often named something generic such as *sample.exe* or *malware.exe* or assigned a hash value like *b3126a1de5401048f5a6ea5a9192126fc7482ff0*. It might also be run from a generic directory such as *C:\Users\<user>\Downloads* or *C:\Users\<user>\Desktop*, rather than the directory the malware author intended, such as a temporary directory. Some malware can identify these anomalies. For example, a malware sample might call the `GetModuleFileName` function to return its own name or the `PathFindFileName` function to return the full path from where it executed. The following sample calls `GetModuleFileName` to test whether its filename is blocklisted:

```
  push  offset size   ; nSize
  push  offset fileNameBuffer  ; lpFilename
  push  esi; hModule (null)
❶ call  GetModuleFileNameA
  --snip--
  loc_21D10:
  push offset blocklist1 ; "malware.exe"
  push fileNameBuffer
❷ call wcsstr
  test eax, eax
  jz   short loc_21E12:
  --snip--
❸ loc_21E12:
  push offset blocklist2 ; "sample.exe"
  push fileNameBuffer
  call wcsstr
  test eax, eax
  jz   short loc_21F10:
```

This malware sample calls the `GetModuleFileNameA` function ❶ with three parameters: a buffer that will store the returned filename for the malware (`fileNameBuffer`), the size of this buffer (`size`), and a target module (`esi`). The target module, when set to `null`, is the currently running process.

Next, the code calls the `wcsstr` function ❷ with two parameters: a blocklisted filename (`blocklist1`) and a pointer to the buffer that stores the mal-

ware filename returned from `GetModuleFileName`. The malware sample then compares its own filename to a blocklist value. If the result is `0`, meaning that the returned filename doesn't match the blocklisted filename, the malware jumps to the next comparison. It continues to iterate through a list of blocklisted filenames ❸. If it finds a match, it assumes it is being analyzed.

Instead of using a generic filename, malware sandboxes and analysts sometimes rename the malware file to its hash value—typically in MD5, SHA-1, or SHA256 format. When you download malware from malware repositories such as VirusTotal, its filename will be in the form of a hash, like *b3126a1de5401048f5a6ea5a9192126fc7482ff0*. The malware analyst or sandbox may simply append the file type extension to this file before running it: *b3126a1de5401048f5a6ea5a9192126fc7482ff0.exe*. Hashes have a set number of characters, such as 32 characters for MD5 hashes and 40 characters for SHA-1 hashes. Malware can count the number of characters in its filename, and if there are exactly 32 or 40 (plus the file extension characters), the malware could assume it's running in an analysis environment.

Additionally, malware can inspect its run path and filename and compare those to the original run path and filename intended by the malware author. For example, a malicious Microsoft Word document might attempt to download an executable file from the internet and save it in the *C:|Users|<user>|AppData|Roaming* directory, with a filename of *abc.exe*. If a malware analyst were to obtain this executable file without the original Word document and run it from the *C:|Users|<user>|Downloads* directory with a filename of *evil.exe*, the malware sample might notice that it was running from an anomalous location with an unknown filename and take evasive actions.

Finally, malware may inspect its command line arguments. Certain sandboxes add their own arguments into the malware process, such as `-force`, `-analysis`, or `-debug`. By checking for the presence of these arguments, malware can detect if it's running inside a sandbox environment and alter its behavior accordingly. A variation of this technique is malware executing only with specific command line arguments. If the malware is executed without them, which might be the case if it's detonated in an automated

sandbox environment, it could cease to fully execute or exhibit different functionalities. For example, the malware sample *evil.exe* may require the command line argument `do_stuff`:

```
C:\> evil.exe do_stuff
```

Without the `do_stuff` argument, the malware may not run correctly in the analysis environment. This technique usually involves a preliminary malware executable or script that executes the primary malware executable with the correct arguments. In this case, you might have to do some manual analysis and reverse engineering to identify the command line arguments the malware is expecting.

To bypass many filename and run path evasion checks that malware might use, you can simply rename the file to a random word or phrase and run the file from a directory that malware is often executed from, such as a temp directory (for example, *C:\Users\<username>\AppData\Roaming*). It's important to understand the full attack chain here too: the malware may be looking for a specific filepath or argument, and a quick way to get this path is to investigate all artifacts of the attack.

Loaded Modules

Certain sandboxes and malware analysis tools load modules into the memory address space of a running malware process to modify the malware's behavior or to intercept its code. In this case, a *module* is typically a DLL file that is loaded (or *injected*) into a target process for various reasons. To determine which modules are loaded in its memory space, the malware can use Windows API functions such as `Module32First`, `Module32Next`, and `GetModuleHandle` to enumerate its loaded modules and identify any anomalies. `Module32First` and `Module32Next` are used to iterate through all the loaded modules inside the calling process, much like `Process32First` and `Process32Next` are used to enumerate processes. `GetModuleHandle`, on the other hand, simply takes a module name as a parameter and returns a handle to the module if it is loaded. Malware can use it to check for a hardcoded list of module names, as shown here:

```
loc_1:
push offset toolSandboxie; "sbiedll.dll"
call GetModuleHandleA
test eax, eax
jz  loc_2:
--snip--
loc_2:
push offset toolVirtualpc; "vmcheck.dll"
call GetModuleHandleA
test eax, eax
```

This malware sample first pushes the address of the string `"sbiedll.dll"` to the stack, then calls `GetModuleHandleA`. If the returned value is `0`, meaning that this module isn't loaded into the malware's memory space, the code jumps to `loc_2`, where it uses the same instructions to check for the `"vmcheck.dll"` module. The two modules this sample is looking for are Sandboxie (*sbiedll.dll*), a popular sandbox application, and *vmcheck.dll*, a module that's often loaded in a Virtual PC guest.

NOTE

This technique can be used for other reasons, like looking for anti-malware, endpoint detection and response (EDR), and other defense tools; we'll discuss this in **Part IV**.

Anomalous Strings in Memory

When hooking or monitoring malware, analysis sandboxes and tools sometimes leave artifacts in the malware's process memory space. Malware can enumerate these strings in memory, searching for specific analysis tools and other suspicious behaviors, as demonstrated by this pseudocode:

```
hashedString = "9253221daaf309200fdcec682a987a51c5a5a598";
ReadProcessMemory(hProcess, lpBaseAddress, lpBuffer, nSize, lpNumberOfBytesRead
hashedBuffer = sha1sum(lpbuffer);
if (memcmp(hashedString, hashedBuffer, sizeof(hashedString)) == 0)
{
```

```
    TerminateProcess();
  }
```

First, this malware sample defines the variable `hashedString`. This string is simply the SHA-1 hash sum of the string `HookLibraryx86.dll`, which I'll discuss in more detail in a moment. Next, the sample will call the `ReadProcessMemory` function with several parameters, the most important of which are a handle to a process (`hProcess`), the base address from which to begin reading memory inside the process (`lpBaseAddresss`), the buffer that will receive the read memory data (`lpBuffer`), and the number of bytes to be read from memory (`nSize`). In this case, the process being read from is the malware's own process. Once the data is read from memory and stored in the buffer, the sample calculates a hash of this data using the SHA-1 algorithm. Finally, the sample calls `memcmp` to compare the hashed buffer data with the original hashed string.

Malware can either search its entire memory address space for anomalies or target a specific memory region. The `lpBaseAddress` value can be any base address within the malware's process address space.

This malware sample's usage of hashing makes analysis more difficult because we can't really know what it's searching for in memory without reversing the hash. *HookLibraryx86.dll* is a module that is commonly loaded into memory when the tool ScyllaHide is being used to analyze malware. I'll cover ScyllaHide in more detail in **Chapter 10**, but for now, keep in mind that malware can scan its own memory for an anomalous string and obfuscate what it's searching for to make analysis more difficult.

Hooked Functions and Acceleration Checks

Automated malware sandboxes and some analysis tools may hook and modify specific functions when running malware to bypass certain evasion attempts. One commonly hooked function is `Sleep`, which malware uses to remain dormant and prevent automated sandboxes from successfully analyzing it (because sandboxes often run for only a few minutes). Sandboxes might modify the parameter to the `Sleep` function to change the sleep time from 5 minutes to 30 seconds, for example. Malware can check whether the `Sleep` function is being tampered with by "sandwich-

ing" it between two timing-based functions, such as `GetTickCount` (this technique is called an *acceleration check*):

```
call GetTickCount
mov edi, eax
push 7530h ; 30,000 ms
call Sleep
call GetTickCount
sub eax, edi
mov ecx, 7148h ; 29,000
cmp ecx, eax
```

This malware sample calls `GetTickCount`, followed by a call to `Sleep` for 30,000 milliseconds (30 seconds). Next, it calls `GetTickCount` a second time, then subtracts the first `GetTickCount` result from the second and stores the difference in `eax` (`sub eax, edi`). Finally, the malware compares the value in `eax` to a value of 29,000 milliseconds, which is the minimum length of time that the malware expects should have elapsed. If an automated sandbox or tool has tampered with the `Sleep` function, the `GetTickCount` values wouldn't be proportionate to this expected length of time, tipping the malware off that something is amiss.

Using Performance and Timing Indicators

There can be significant timing and performance deviations between real, non-virtualized processors and their virtualized counterparts. Non-virtualized processors typically execute instructions more quickly and more efficiently than a virtual processor using the same hardware specifications. Most of the time, this isn't noticeable to the humans interfacing with these systems. Malware, however, can exploit these slight differences to reveal the underlying VM.

The rdtsc Instruction

The *Read Time-Stamp Counter*, or `rdtsc`, is a special assembly instruction with a number of use cases, including performance monitoring and metrics gathering. When the `rdtsc` instruction is executed on a CPU, the current number of CPU ticks is stored and can be referenced and compared

to a later `rdtsc` instruction. The difference between the two values is the number of ticks since the first `rdtsc` instruction.

VMs have a problem with the `rdtsc` instruction. When `rdtsc` is executed by a program running on a virtual processor, the virtual processor must pass this instruction on to the "real" processor: the hardware CPU that runs the host OS. Passing the `rdtsc` instruction to the real CPU and back again causes some latency, which in turn increases the number of ticks. When malware sees that this returned tick count is higher than it would be on a non-virtualized system, it might deduce that it's running inside a VM environment.

To better understand the way malware can use this technique, let's look at an excerpt of some assembly code from a disassembled piece of malware:

```
rdtsc
mov [ebp+rdtsc_1], eax
xor eax, eax
rdtsc
sub eax, [ebp+rdtsc_1]
```

The malware sample uses the `rdtsc` instruction to get the current tick count and stores this value in a buffer (`rdtsc_1`). Then, it executes another `rdtsc` instruction and subtracts the result from the original (`sub eax, [ebp+rdtsc_1]`). It can use the resulting value to determine whether the execution time is within the threshold of a non-virtualized processor.

Sometimes malware gets craftier than two simple `rdtsc` commands, and some malware families execute multiple iterations of `rdtsc` to get a more accurate value. In the next section, we'll cover a few more use cases for `rdtsc`.

Function Execution Timing

Given the timing differences between non-virtualized and virtualized processors, some malware runs a function or instruction and compares its execution time to a baseline representing a real, non-virtualized pro-

cessor. If the instructions execute more slowly than the baseline, the malware could determine that it's running in a VM.

One technique is for the malware to sandwich a Windows API function call or instruction between two `GetTickCount` functions and compare the results. A long delay between the two `GetTickCount` functions could signify to the malware that it's being executed and analyzed.

There are many variations of this technique, including using instructions such as `rdtsc` and `cpuid` instead of Windows API functions such as `GetTickCount`. The takeaway here is that any time you spot malware using timing-based functions or instructions, followed by a comparison operation, you should investigate it in more detail.

Performance Counters

The CPU maintains *performance counters* to represent the number of instructions that have executed since a certain point in time. They can be used legitimately to gather performance metrics, but malware can use them to detect VM environments. The `QueryPerformanceCounter` function queries the processor's performance counter and returns the current value. `QueryPerformanceFrequency` returns the performance counter's *frequency*, a fixed value representing the processor's overall performance. When these instructions are executed in a VM, the values returned by these functions may be slightly lower than on a non-virtualized system, demonstrating the performance impact of virtualized processors.

If you spot malware using these functions, look for a subsequent comparison operation. This may be malware attempting to identify your analysis environment.

Abusing the Virtual Processor

The virtual processors inside a VM may interpret and execute certain assembly instructions in a way that reveals the VM's presence to malware. In this section, we'll delve into how malware "abuses" these virtual processors by misusing instructions or exploiting their design flaws.

The Red Pill and No Pill Techniques

The Red Pill and No Pill techniques are two well-known methods used by malware to detect virtual environments. Both techniques check processor data structures for VM-specific values. These techniques are often ineffective in modern analysis environments, so they're seldom used in modern malware, but it's still good to be aware of them.

For the Red Pill technique, malware executes the *Store Interrupt Descriptor Table*, or `sidt`, assembly instruction, which writes the value of the Interrupt Descriptor Table register to memory, then checks it. The *Interrupt Descriptor Table (IDT)* is a data structure the processor uses to determine the correct response to exceptions and interrupts. (*Exceptions* signal to a processor that something is wrong with an instruction, and *interrupts* allow the processor to respond to a higher-priority event when required.) If the malware is running in a VM, the fifth byte of the IDT register will contain a specific value indicating that.

The Red Pill technique is simple to implement:

```
lea  eax, [ebp+buffer]
sidt [eax]
mov  al, [eax+5]
cmp  al, 0FFh
```

The `sidt [eax]` instruction stores the contents of the IDT register, which is 6 bytes long, to a buffer. The code reads this value and executes a comparison operation to the fifth byte of the buffer, `[eax+5]`. The fifth byte of the IDT register will be equal to `FF` (in hexadecimal) if it's running in a VM.

The No Pill technique uses the *Store Local Descriptor Table*, or `sldt`, instruction to store the *Local Descriptor Table* (*LDT*) register to memory. Since normal Windows applications don't use the LDT, this LDT register should have a value of `0`. However, on some hypervisors, this LDT register value will contain a nonzero value, which can be a hint to the malware that it's running in a VM. Similarly to the Red Pill technique, the No Pill

technique doesn't work correctly anymore in modern sandboxes and hypervisors.

IO Ports

An *input/output port (IO port)* is a communication method between the physical hardware of the machine and the software that's running on it. Certain hypervisors use IO ports for host-to-guest OS communication. VMware, for example, uses the VMX IO port. Research from Cisco Talos revealed that malware can identify the VMX port to detect whether it is running in a VMware environment. The following assembly code shows what this might look like in practice:

```
mov   eax, 'VMXh'
mov   ebx, 2EF36D4Ch
mov   ecx, 0Ah
mov   dx, 'VX'
in    eax, dx
cmp   ebx, 2EF36D4Ch
```

This malware sample loads the value of `VMXh` into the `eax` register, then loads the magic value of `2EF36D4Ch` into the `ebx` register. This magic value can be any hexadecimal value, and it doesn't matter here. Next, the code loads the `dx` register segment with the value of `VX`, or `5658` in hexadecimal, which is the VMX port number. Finally, the malware executes `in eax, dx`, which uses the `in` assembly instruction to attempt to access the IO port. If this host is running the VMware Workstation hypervisor, this instruction will return the magic number, which will likely trigger the malware sample to terminate itself or take other evasive actions. On a non-virtualized host, the returned value would be `0`.

As with the Red Pill and No Pill techniques, this specific technique is quite old and has already been patched in modern versions of VMware. However, it's important to understand this technique and others like it in case you're analyzing malware that uses it or some variation of it.

The cpuid Instruction

The `cpuid` assembly instruction returns information about the host's processors, such as the processor's features and manufacturer. Outside of a VM, executing `cpuid` with EAX set to `0` will return something like `Genuineintel` for Intel processors or `AuthenticAMD` for AMD processors. When `cpuid` is executed inside a VM, it often returns the name of the hypervisor. In the case of VMware, this string is `VMwareVMware`. For VirtualBox, the returned string is `VBoxVBoxVBox`.

Setting EAX to `1` prior to executing `cpuid` returns much more information, stored as a 31-bit block in the ECX and EDX registers. The 31st bit in the ECX register will be `0` if the system's CPU is physical or `1` if the system is using a virtual CPU, indicating a VM environment. Here's an excerpt of assembly code from a malware sample using this technique:

```
inc eax
cpuid
bt ecx, 0x1f
jc terminate_process
```

Here the malware sets `eax` to `1` (`inc eax`), which will be used as a parameter for the `cpuid` instruction following it. After `cpuid` is executed, the malware executes the `bt` (*bit test*) instruction to move the 31st bit of `ecx` into the *carry flag* register, which is a special CPU register that can store values of `0` or `1` and is typically used during addition and subtraction operations. Finally, the sample checks the carry flag register (`jc`), and if it's set to `1`, the malware will conclude that it's running in a VM and terminate itself.

NOTE

Wikipedia is a great reference for the processor information and feature bits returned from `cpuid`. See https://en.wikipedia.org/wiki/CPUID.

Unsupported Instruction Sets

Malware may use `cpuid` to find out whether a specific instruction set is supported by the processor. For example, the 23rd bit within the EDX reg-

ister specifies whether the processor supports the *SSE instruction set*, a set of uncommon assembly instructions that's typically used for graphics processing and scientific computing. Some modern hypervisors support instruction sets like SSE, but not all do! This can be a dead giveaway for malware that it's running in a VM or sandbox.

Some malware, instead of methodically looking through the output of `cpuid`, will simply try to execute these instructions, like so:

```
movdqa xmm0, xmmword ptr [eax]
movdqa xmm1, xmmword ptr [eax]
movdqa xmmword ptr [eax], xmm0
movdqa xmmword ptr [eax], xmm1
```

The `movdqa` (Move Aligned Double Quadword) instruction is used for moving data to and from *XMM registers*, which are those used in the SSE instruction set. In this code, the `movdqa` instruction moves the data referred to by `eax` to the `xmm0` register and again to the `xmm1` register. Then, it moves this data back from the `xmm` registers to `eax`. If this code produces an error (or the malware crashes!), the malware sample may assume it's being run in a virtualized environment.

The SSE instruction set is just an example here, and most modern hypervisors support it. The important thing to remember is that any exotic or uncommon assembly instructions can be abused by malware for VM and sandbox detection if the hypervisor doesn't support them.

Another example of an exotic instruction is `vpcext`, which produces an error if executed outside of the Windows Virtual PC hypervisor. If executed inside Virtual PC, however, the instruction will succeed and set the EBX register to `0`. The following code executes the `vpcext` instruction, followed by a `test` instruction to check whether `ebx` is equal to `0`:

```
vpcext 7, 0bh
test ebx, ebx
```

The Trap Flag and Other Techniques

The trap flag is the eighth bit in the EFLAGS register in the Intel x86 instruction set. If it is enabled prior to another instruction call, an exception will be triggered. In a VM environment, the hypervisor emulates the trap flag behavior. Depending on the hypervisor in use, this emulation may not be correct or complete, which will cause the trap flag to be ignored and notify the malware that it's running in a VM. This technique was first discovered in the wild in 2021 by researchers at Palo Alto's Unit 42.

It seems that new VM detection methods like this trap flag technique are discovered every few years. Most of them have one thing in common: they abuse the way the CPU and its architecture were designed to function, causing the hypervisor to behave in unexpected or previously unknown ways and ultimately exposing the underlying VM. Malware analysts must keep up to date on these techniques to identify when malware is using them.

The Risks of Using Detection Techniques

As you've seen in this chapter and throughout the previous chapters, for malware authors there can be many benefits of implementing VM and analysis tool detection functionalities. So why might a malware author *not* include them? One reason is that these techniques can actually increase the chance of the malware being detected by anti-malware or an analyst. The more suspicious the Windows API functions or instructions the malware executes, the greater the chance that these behavioral anomalies will be noticed.

Another risk is less obvious and, in my opinion, a bit comical. If malware detects that it's running in a VM and subsequently chooses not to infect the host, this can be self-defeating in certain circumstances. Many organizations are switching to cloud and on-demand infrastructure, where many systems are in fact VMs. Malware that can detect VMs to evade malware analysts and sandboxes often is simultaneously evading the exact corporate systems it's designed to infect.

One real-world example is the well-known cybercrime malware Emotet, which featured several anti-analysis techniques, one of which was VM and sandbox detection. If it deemed the host a VM or sandbox, Emotet would either terminate itself, thus preventing infection, or behave differently from how it otherwise would on a real, physical system. This led to a lower Emotet infection rate than there would have been otherwise and likely saved some organizations running on virtualized infrastructure from major harm.

Summary

In this chapter, you learned about several techniques that malware can use to establish context and detect malware analysis tooling, VMs, and sandboxes. Next, we'll build on what we've discussed in this and the previous three chapters. Specifically, we'll examine what malware does to evade and disrupt analysis efforts when it discovers it's running in a virtualized environment or lab.